# Maintainer Prioritization in npm Supply Chains: Coverage, Exposure, and Graph Representation

Anonymous Authors

***Abstract*—Limited defensive budgets motivate prioritizing npm maintainers by downstream reach. However, reaching many repositories and reducing exposure to possible account compromises are different objectives. We study this distinction in our collected graph of 25,228 package nodes and 12,217 maintainer usernames. We compare five deterministic selection methods and 200 random orders across six budgets, using package and workflow-associated repository endpoints. At 100 identities, marginal coverage greedy reaches 5,538 of 7,171 observed workflow repositories (77.2%), versus 4,407 for exact maintainer-union ranking. Under a stipulated uniform single-account compromise model with perfect source hardening, their exposure reductions are 11.48% and 33.79%, respectively. Only 800 repositories covered by greedy satisfy exclusion of every observed source identity. Preserving version continuity reduces identity–repository exposure pairs by 42.44%, although aggregate greedy coverage remains similar. These results extend established maintainer-reach analyses with an explicit comparison of objectives and observation units. They support objective-specific prioritization and qualified structural measurements; they do not estimate incident probabilities or demonstrated control effectiveness.**

***Index Terms*—software supply chain, npm, maintainer prioritization, dependency graphs, workflow exposure, empirical study**

## I. Introduction

Shared dependencies concentrate software supply-chain exposure in a relatively small number of upstream packages and maintainer accounts. Prior npm research demonstrates both this concentration and the potential reach of selected maintainers [1], [2]. A registry operator or security team with a limited budget must nevertheless decide what the selection should accomplish. Reviewing distinct downstream repositories, reducing expected exposure across hypothetical account compromises, and eliminating every modeled source of exposure at a repository require different measurements.

Consider two accounts whose packages reach nearly identical downstream repositories. Selecting both can be valuable when each account represents an independent possible origin of compromise. The second account contributes little additional repository coverage, however. Conversely, a selection that broadens coverage can leave most covered repositories reachable through unselected accounts. A coverage percentage therefore needs an explicit interpretation before it can inform an account-hardening decision.

We investigate this distinction through a reproducible structural study of our npm data. We construct maintainer portfolio unions, deduplicate workflow-associated packages into repository endpoints, and evaluate established selection rules against three separately defined quantities: distinct coverage, additive scenario exposure reduction, and exclusion of all observed source identities. We then test whether the conclusions depend on collapsing package versions, counting monorepo packages separately, limiting graph depth, or changing visibility and selection assumptions.

Our study addresses three questions. **RQ1:** How do coverage-oriented and additive exposure-oriented selections differ at the same identity budget? **RQ2:** How much covered infrastructure remains reachable through unselected identities? **RQ3:** How sensitive are the results to graph representation, endpoint units, and observation gaps?

The contribution is an empirical replication and extension of maintainer-reach analysis. First, we compare five deterministic methods and a 200-order random baseline at six budgets, with exact graph closures and explicit objectives. Second, we quantify workflow-repository deduplication and version continuity effects on exposure mass and coverage. Third, we provide aggregate reproducibility records, independent correctness checks, and a frozen prospective acquisition pilot for subsequent outcome research. At 100 identities, coverage greedy reaches 77.2% of observed workflow repositories but removes 11.5% of exposure under our equal-account model; exact-union ranking removes 33.8%. This divergence motivates specifying the decision objective before interpreting a ranking as defensive benefit.

## II. Related Work

**Maintainer concentration and prioritization.** Zimmermann et al. [1] define maintainer reach as a union over maintained packages, formulate greedy maintainer coverage, and study maintainer vetting. These establish the central reach and coverage techniques used here. Zahan et al. [2] examine npm weak-link signals, including maintainer characteristics. Wang's recent preprint [3] evaluates centrality-based package-node hardening using cascade simulations. Our experimental unit is an observed maintainer username, and our intervention removes only the selected account-origin scenario. It does not delete a package or halt traversal through a hardened downstream package. We extend these studies by jointly reporting coverage, additive scenario reduction, and all-source exclusion under the same selections.

**Coverage and dependency semantics.** Cardinality-constrained monotone submodular maximization and the greedy approximation guarantee are established [4]. Our exact-union ranking optimizes a separate modular objective under equal scenario weights and costs. Neither algorithmic property is a novelty claim. Liu et al. [5] show the importance of npm

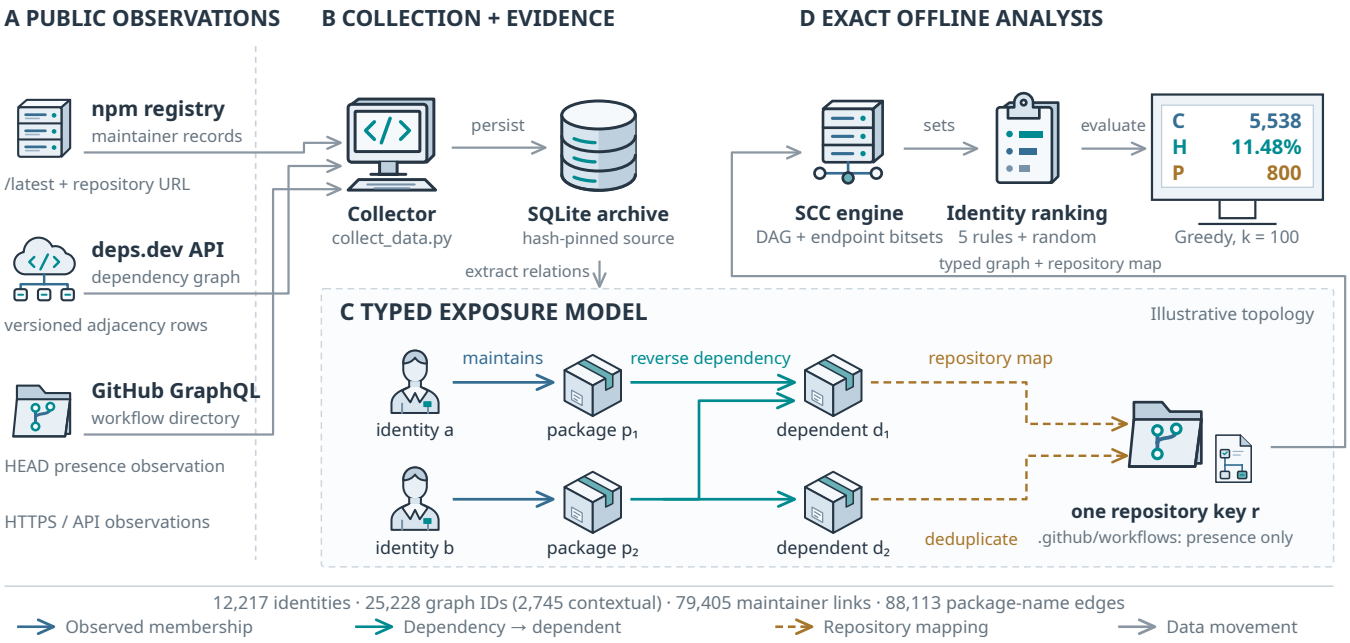



Fig. 1. Implemented measurement architecture. Device symbols denote logical roles; the typed topology is illustrative. Colored arrows distinguish observed maintainer links, dependency-to-dependent exposure, and repository mapping. Counts describe our sample. The display reports greedy selection at $k = 100$: coverage $C = 5{,}538$, conditional exposure reduction $H = 11.48\%$, and all-source exclusion $P = 800$, under the stated source-hardening model.

resolution semantics for dependency analysis. He et al. [6] quantify counterfactual version-pinning effects using historical npm resolution. We evaluate package projection against version-continuous paths while distinguishing an observed union graph from a resolved installation.

**Workflow and temporal evidence.** Workflow security characterization [7] and ARGUS taint analysis [8] examine configuration or data flow beyond directory presence. Our flag identifies associated endpoints without equivalent evidence of execution or secret access. Tsakpinis and Pretschner [9] report repository-accessibility differences across package importance, supporting explicit treatment of observation gaps. npm-follower [10] and CHRONO-RESOLUTION [11] support archival and release-point analysis; PatchRisk [12] studies future vulnerability exposure. These motivate temporal follow-up, but our cross-sectional experiment evaluates structural objectives rather than forecasting compromise. Our literature search through 20 October 2026 positions the contribution as this measured extension, without claiming the first npm graph analysis or forecasting method.

## III. Data and Graph Construction

### A. Collection and Observation Scope

Our collection pipeline (Fig. 1) seeds a queue from an npm high-impact package list, expands dependencies through the deps.dev API, retrieves npm registry metadata, and checks linked GitHub repositories for a `.github/workflows` directory at HEAD. The sample contains 25,228 package identifiers, 22,340 package-version metadata rows, 12,217 maintainer usernames, and 79,405 package–maintainer relations. Queue insertion timestamps span 13–16 June 2026; they do not establish a common acquisition cutoff for every field. We analyze the frozen archive on 20 October 2026 and retain its checksum in the execution manifest. The identifiers include 2,745 bundled-context names emitted by deps.dev; they are graph nodes rather than separate global registry coordinates [13]. They have no registry metadata in our collection. We retain their context-qualified identities instead of aliasing them to global leaf names. This selected expansion is not a random sample of all npm packages.

The dependency data contain 121,885 directed version-edge rows and 88,113 distinct package-name adjacencies. We use explicit adjacency entries from dependency responses [13]. A stored target-node `DIRECT`/`INDIRECT` label describes its relationship to the original resolution root; filtering edges by that label would not recover a global graph of direct adjacencies. We reverse declaring-to-dependency edges to measure downstream reach, retaining all explicit adjacencies.

### B. Endpoints and Missing Evidence

We evaluate three endpoint universes: all package names, packages with any positive workflow observation, and distinct repository keys associated with those positive observations. The 15,233 positive packages map to 7,171 repositories. Keys are case-folded owner/name pairs with a trailing transport `.git` removed. This lexical mapping deduplicates shared repositories; it does not verify redirects, commits, jobs, or independent CI installations.

There are 3,081 packages without an observed maintainer mapping and 3,772 without a completed workflow flag. A stored zero flag can conflate an absent directory with an

unavailable repository. We therefore use positive observations for workflow endpoints and do not treat zero or missing values as proof of safety or workflow absence. Of the 7,171 positive repositories, 7,158 are reachable from an observed maintainer portfolio. Coverage percentages use all 7,171; all-source exclusion requires a nonempty observed source set.

### C. Two Graph Representations

Our primary graph collapses retained versions into package-name nodes. For sensitivity analysis, we construct 40,806 exact (package, version) nodes and permit traversal only when adjacent version keys match. Each maintained package seeds every version present in that graph; packages without a version node retain only their eligible seed endpoint. Reach includes maintained seeds, and the same positive endpoint definition is used for both representations. Version continuity can remove paths created by joining incompatible versions, but both graphs combine retained resolution contexts and acquisition times. The version graph is a sensitivity representation, not a historical installation or ground truth.

## IV. Objectives and Experimental Method

### A. Maintainer Exposure Sets

Let $M$ be the observed usernames and $P_m$ the packages linked to $m$. For an endpoint universe $U$, we construct

$$E_m = \bigcup_{p \in P_m} \mathcal{D}_U(p), \qquad A_e = \{m \in M : e \in E_m\}, \quad (1)$$

where $\mathcal{D}_U(p)$ contains reverse-reachable endpoints of $p$, including its eligible seed endpoint. Repository endpoints are deduplicated across every package and path. We condense strongly connected components and propagate exact integer bitsets through the resulting directed acyclic graph. A separate propagation of source-identity bitsets checks the endpoint incidence relation.

### B. Three Quantities for the Same Selection

For selected identities $S \subseteq M$, distinct endpoint coverage is

$$C(S) = \left| \bigcup_{m \in S} E_m \right|. \quad (2)$$

It measures the breadth of structural scenarios represented by a review or control selection. Overlap across identities contributes once.

To model source-account hardening, suppose exactly one observed username is selected uniformly as a hypothetical compromise origin. Assume a perfect control prevents that origin when selected, while all other scenarios and graph paths remain unchanged. The proportional reduction in expected endpoint count is then

$$H(S) = \frac{\sum_{m \in S} |E_m|}{\sum_{m \in M} |E_m|}. \quad (3)$$

The fixed uniform prior cancels in this ratio. It is a stipulated weighting, not an estimated account-compromise probability. Descending exact $|E_m|$ is optimal for $H$ under equal costs because its numerator is additive. Hardening does not remove a node or protect other origins passing through its packages.

Finally, we count endpoints for which every observed source has been selected:

$$P(S) = \sum_{e \in U} \mathbf{1}[A_e \neq \varnothing \ \wedge \ A_e \subseteq S]. \quad (4)$$

We call this *all-source exclusion*. Endpoints without observed sources are excluded rather than counted as protected. Under the idealized model, $C(S) - P(S)$ covered endpoints remain reachable from an unselected source. Neither quantity establishes real control effectiveness.

The distinction appears in a small example. For exposure sets $\{1, 2, 3, 4\}$, $\{1, 2, 3\}$, and $\{5, 6\}$, coverage greedy selects the first and third at budget two, covering six endpoints with six selected identity–endpoint pairs. Exact-union ranking selects the first and second, covering four endpoints with seven pairs. The objective determines the preferred selection.

### C. Selection Rules and Budget Grid

We compare portfolio size, summed per-package reach, maximum per-package reach, exact portfolio-union reach, and greedy maximum marginal coverage. The sum baseline double-counts overlap within a portfolio. Greedy repeatedly adds the identity contributing the most newly covered endpoints. Coverage is monotone submodular, so this established method has the standard $1 - 1/e$ lower bound relative to optimal coverage [4]; that guarantee does not apply to $H$ or $P$. We do not optimize $P$ globally.

Selections are nested at budgets $1, 5, 10, 25, 50, 100$, with equal identity costs. Deterministic ties use ascending username order. We sample 200 seeded uniform orders without replacement, reusing each order across endpoint universes, and calculate exact random expectations from source multiplicities using hypergeometric probabilities. Random central 95% simulation ranges describe selection variability on this graph, not confidence intervals for npm or real attacks. Our saved analysis plan is exploratory and recorded before this execution; it is not an external preregistration.

### D. Reproducibility and Checks

The read-only implementation uses Python's standard library and records source/output hashes, seeds, environment, and definitions. We verify 288 graph roots independently by breadth-first search, match all 25,228 stored package transitive counts, and check source/endpoint duality, union bounds, nested curves, and $P \leq C$. Separate tests cover the objective counterexample, cycles, version splicing, and 60 seeded small graphs comprising 675 independently traversed roots. The study evaluates 150 deterministic and 6,000 random budget rows across five endpoint/visibility definitions. No package payload is executed.

Three isolated full reruns reproduce the aggregate results byte for byte. On our Linux x86-64 host with Python 3.14.8, wall time averages 39.50 seconds (sample standard deviation

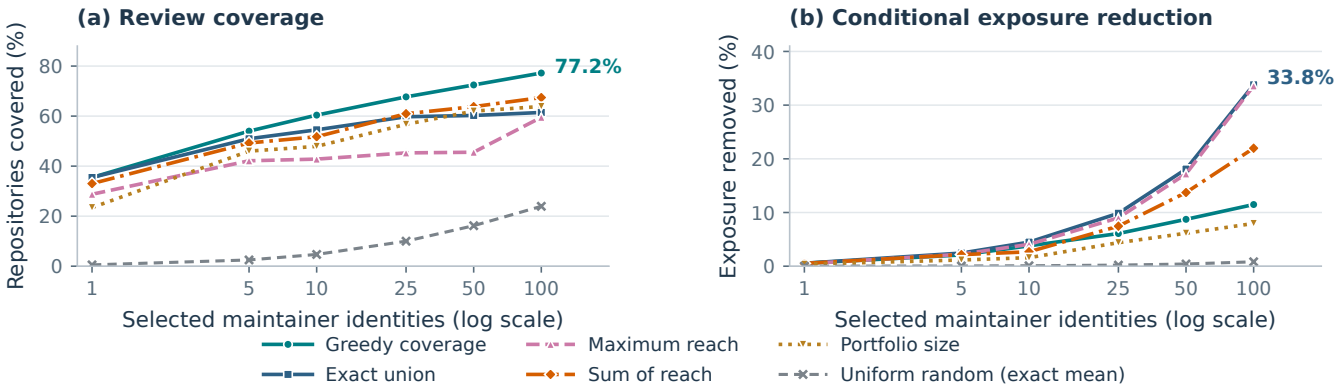



Fig. 2. Workflow-repository coverage and conditional exposure reduction favor different selections. Lines connect the six evaluated budgets. The random baseline shows exact finite-population expectations, without confidence bands.

TABLE I
WORKFLOW-REPOSITORY RESULTS AT 100 IDENTITIES. $C$: DISTINCT COVERAGE; $H$: CONDITIONAL SCENARIO EXPOSURE REDUCTION; $P$: ALL-SOURCE EXCLUSION. RANDOM VALUES ARE 200-ORDER MEANS.

| Selection | $C$ | $C$ (%) | $H$ (%) | $P$ |
|---|---|---|---|---|
| Portfolio | 4,579 | 63.9 | 7.96 | 577 |
| Summed reach | 4,835 | 67.4 | 21.98 | 704 |
| Maximum reach | 4,268 | 59.5 | 33.59 | 533 |
| Exact union | 4,407 | 61.5 | 33.79 | 534 |
| Greedy coverage | 5,538 | 77.2 | 11.48 | 800 |
| Random (mean) | 1,680.6 | 23.4 | 0.80 | 12.8 |

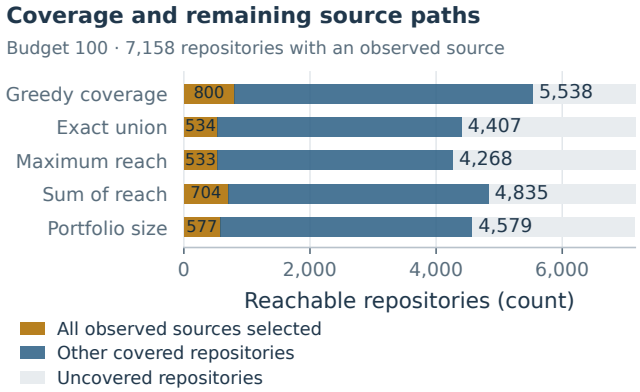



Fig. 3. Repository source-set partition at budget 100. Segment labels count $P$; boundary labels count $C$. Bars exclude 13 repositories without observed sources. Blue segments remain reachable through unselected identities.

0.98 seconds), with maximum process peak RSS of 1,038.4 MiB. These descriptive measurements include extraction, every baseline and sensitivity, and output serialization. Filesystem caches are uncontrolled; they do not estimate per-package inference or online collection cost.

## V. RESULTS

### A. RQ1: Coverage and Exposure Favor Different Selections

Table I reports workflow-repository outcomes at 100 identities (0.819% of usernames). Greedy covers 5,538 repositories (77.2%), versus 4,407 (61.5%) for exact union: a descriptive difference of $\Delta C = +1{,}131$ repositories (+25.7%). Exact union instead removes 33.79% of stipulated exposure, versus 11.48% for greedy ($\Delta H = -22.30$ percentage points; $\Delta P = +266$). Its $H$ advantage follows analytically; the experiment quantifies the resulting coverage tradeoff. Maximum reach nears this $H$ but covers fewer repositories.

Figure 2 shows budget curves. At 25 identities, greedy covers 4,854 repositories, versus 4,284 for exact union and 4,067 for portfolio size ($H = 6.08\%, 9.86\%, 4.38\%$). At 100, random mean coverage is 1,680.6 (sample SD 530.7, Monte Carlo standard error $\text{MCSE}_C = 37.5$), with a central 95% simulation range of 659.8–2,610.7. The exact finite-population expectation is $E[C] = 1{,}716.8$; expected $H$ is $100/12{,}217 = 0.819\%$ (SD 0.373 pp, $\text{MCSE}_H = 0.026$ pp). Differences reflect random-order variation, not sampling inference about npm.

### B. RQ2: Covered Endpoints Retain Other Sources

Figure 3 partitions source-reachable repositories at budget 100. Of the 5,538 repositories covered by greedy, only 800 meet all-source exclusion; 4,738 remain reachable from an unselected identity. Across the 7,158 repositories with nonempty observed source sets, 6,358 retain such a source. Exact-union ranking excludes every observed source for 534 repositories. We make no claim that either selection optimizes $P$.

Cross-identity overlap explains the difference between broad coverage and additive scenario mass. At 100 identities, $1 - C(S)/\sum_{m \in S} |E_m|$ is 89.57% for greedy and 97.18% for exact union. An observed co-maintenance pattern illustrates this masking without asserting a historical forecast: `qix` maintains only three packages (`debug`, `simple-swizzle`, `backslash`; portfolio rank #2,298) yet reaches 2,971 repositories, ranking #8 under exact union and #5 under version continuity (including 444 repositories not reached by `sindresorhus`). Because `sindresorhus` (#1) and

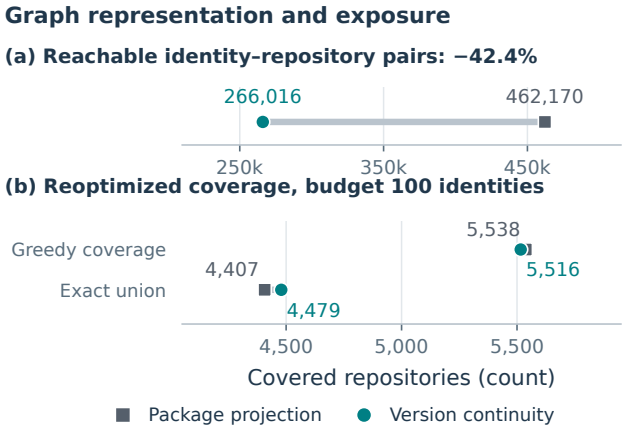



Fig. 4. Version continuity changes pair mass while aggregate coverage remains similar. The lower panel selects and evaluates within each representation at budget 100. Both graphs remain observed unions.

`tootallnate` (#8) absorb 2,527 of those endpoints, greedy marginal coverage skips `qix` past rank 100 while leaving all 2,971 repositories residually exposed.

### C. RQ3: Representation Changes Exposure Mass

Preserving version continuity changes downstream closures for 8,587 of 25,228 package roots and removes 1,273,291 ordered root–dependent pairs introduced by package projection. Maintainer workflow-repository unions change for 2,582 usernames. Total identity–repository pairs fall from 462,170 to 266,016, a 42.44% decrease. This is representational sensitivity, not an estimate of false exploit paths in actual installations.

At 100 identities, selecting and evaluating greedy on the version graph covers 5,516 repositories, with $H = 13.10\%$ and $P = 817$. Exact-union ranking gives $C = 4,479$, $H = 40.59\%$, and $P = 527$. Evaluating the package-selected greedy identities on that version graph yields $C = 5,491$. Aggregate coverage remains similar at this budget, while additive exposure mass is substantially representation-dependent (Fig. 4).

Repository deduplication affects decisions. Selecting greedy for all-package coverage, then evaluating repository endpoints, covers 5,308 repositories; selecting for workflow-package coverage gives 5,315. Direct repository selection gives 5,538. Depth ablation gives 5,255 when selection uses portfolio endpoints alone and 5,488 with one-hop evidence, both evaluated against full reach. Endpoint units and depth therefore matter with the final evaluation denominator held fixed.

## VI. Robustness and Research Implications

### A. Observation and Selection Sensitivities

We exclude 60 positive observations whose stored repository key disagrees with a lexical parse of the source URL, reducing the endpoint universe by 24 repositories (7,171 to 7,147); greedy at 100 covers 5,524, with $P = 799$. We do not transfer workflow flags without reacquisition. This check addresses a specific mapping defect and cannot certify remaining remote associations.

Removing all bundled-context nodes leaves every workflow-repository source set unchanged, including all 462,170 identity–repository pairs. Consequently, primary repository $C$, $H$, and $P$ are identical for any fixed selection. Workflow-package incidence changes by 28 pairs for one endpoint, which repository deduplication absorbs; contextual nodes should therefore not be assumed irrelevant to every endpoint definition.

Reversing tie order yields $C = 5,537$ and $P = 793$. Excluding the 1, 5, or 10 largest portfolios gives coverage of 5,522, 5,149, and 5,076. Grouping equal stored email hashes as a hypothetical control unit yields 12,147 groups with greedy coverage at 5,538. These tests reflect eligibility sensitivity rather than verified credential control.

### B. From Structural Reach to Outcome Research

Our findings suggest distinct uses for the quantities. A team seeking broad review coverage can use $C$; the stipulated fixed-prior account-hardening decision uses $H$. Complete modeled source exclusion requires $P$ and may demand complementary selections. Actual budgets also depend on control cost, feasibility, account-specific probabilities, publishing permissions, and adversary adaptation. Our equal-cost fixed-prior experiment does not observe these. Workflow association alone cannot establish credential-harvesting return or a control's causal effect.

We completed prospective acquisition groundwork by sampling 100 identifiers uniformly without replacement from our 25,228-name frame (seed 20261020, frozen before requests). Metadata were captured for 90; eight HTTP 405 responses involve bundled identifiers, while two return HTTP 404. The feature cutoff is 20 October 2026, 00:48:38 Asia/Dhaka, with a 90-day horizon ending 18 January 2027. All 100 outcomes remain pending. This pilot demonstrates feasibility; its size does not establish event yield or predictive power.

Future compromise evaluation must fix surveillance sources, reporting delays, censoring, and family-grouped splits before examining outcomes. Unreported packages remain unlabeled rather than certified benign. Contemporary metadata cannot be retroactively assigned to earlier attacks. We report no trained forecast, accuracy, or empirical compromise probability from this pilot.

## VII. Validity, Limitations, and Responsible Reporting

**Construct validity.** Observed maintainer membership does not establish active publishing authority, account independence, token scope, or control feasibility. Graph reach does not prove that a dependency is installed, executed in CI, or able to access secrets. Perfect source hardening and a uniform origin prior define a conditional comparison; they do not describe measured attack behavior. Real adversaries could adapt to controls.

**External and observation validity.** Collection lacks a shared field-level cutoff, workflow commit identities, and extraction-root provenance. Even version continuity can mix resolution contexts. Missing metadata and inaccessible repositories can distort endpoints and source membership; zero workflow flags remain ambiguous. Our sensitivities describe observable changes rather than correcting every missing value. The high-impact-seeded sample limits ecosystem-wide generalization.

**Internal validity and reproducibility.** Independent traversals, dual incidence checks, toy cases, and generated tables reduce implementation error. Source archives remain unchanged. We recompute from graph relations and record earlier export inconsistencies as quality-control findings, rather than mixing incompatible scores.

**Conclusion validity.** Deterministic comparisons describe this snapshot and have no sampling confidence interval. Random orders quantify baseline-selection variability only. Budgets and endpoint definitions reuse those orders; the 6,000 budget rows are not independent samples.

**Responsible reporting.** We report aggregates rather than individual maintainer priorities or executable attack instructions. We execute no npm payload, test no credentials, and claim no real-world compromise or prevention. Prospective captures retain sanitized metadata and response hashes without emails or package bodies.

## VIII. Conclusion

Our npm study measures a substantial tradeoff between distinct downstream coverage and additive exposure reduction under a fixed account-origin model. At 100 identities, greedy covers 5,538 workflow repositories but excludes every observed source for only 800; exact-union ranking yields greater conditional exposure reduction. Version continuity reduces total exposure pairs by 42.44% while leaving aggregate greedy coverage comparatively stable. These measurements extend established maintainer-reach methods with explicit objective and representation comparisons. The completed prospective acquisition pilot provides a frozen starting point for outcome research; pending observations support no forecast-performance claim.

## Acknowledgment

The authors thank the npm package maintainers and the open-source community for making the software artifacts studied in this work publicly available. We also acknowledge the npm registry, deps.dev, and GitHub for providing the metadata and interfaces used for data collection. In accordance with venue policy, the authors declare that a large language model (ChatGPT / LLM) was used in the preparation of this manuscript solely for non-intellectual editing and grammatical polishing. All generated text was reviewed and approved by the authors, who retain sole responsibility for the content and accuracy of this work.

## References

[1] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *28th USENIX Security Symposium*. USENIX Association, 2019, pp. 995–1010. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman

[2] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams, "What are weak links in the npm supply chain?" arXiv:2112.10165v2, 2022, ICSE SEIP 2022; DOI: 10.1145/3510457.3513044. [Online]. Available: https://arxiv.org/abs/2112.10165v2

[3] Z. Wang, "Defending the core: A centrality-based protection strategy for supply chain security in npm dependency network," arXiv:2606.14036, 2026, preprint. [Online]. Available: https://arxiv.org/abs/2606.14036

[4] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—I," *Mathematical Programming*, vol. 14, pp. 265–294, 1978.

[5] C. Liu, S. Chen, L. Fan, B. Chen, Y. Liu, and X. Peng, "Demystifying the vulnerability propagation and its evolution via dependency trees in the NPM ecosystem," in *44th International Conference on Software Engineering*, 2022. [Online]. Available: https://arxiv.org/abs/2201.03981v2

[6] H. He, B. Vasilescu, and C. Kästner, "Pinning is futile: You need more than local dependency versioning to defend against supply chain attacks," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, 2025, article FSE013. [Online]. Available: https://arxiv.org/abs/2502.06662

[7] I. Koishybayev, A. Nahapetyan, R. Zachariah, S. Muralee, B. Reaves, A. Kapravelos, and A. Machiry, "Characterizing the security of GitHub CI workflows," in *31st USENIX Security Symposium*. USENIX Association, 2022, pp. 2747–2763. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev

[8] S. Muralee, I. Koishybayev, A. Nahapetyan, G. Tystahl, B. Reaves, A. Bianchi, W. Enck, A. Kapravelos, and A. Machiry, "ARGUS: A framework for staged static taint analysis of GitHub workflows and actions," in *32nd USENIX Security Symposium*. USENIX Association, 2023, pp. 6983–7000. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/muralee

[9] A. Tsakpinis and A. Pretschner, "Analyzing the accessibility of GitHub repositories for PyPI and NPM libraries," in *28th International Conference on Evaluation and Assessment in Software Engineering*, 2024. [Online]. Available: https://arxiv.org/abs/2404.17403v2

[10] D. Pinckney, F. Cassano, A. Guha, and J. Bell, "npm-follower: A complete dataset tracking the NPM ecosystem," arXiv:2308.12545, 2023. [Online]. Available: https://arxiv.org/abs/2308.12545

[11] I. Rahman, J. Marley, R. Paramitha, and L. Williams, "CHRONO-RESOLUTION: A dependency resolution dataset at release points for npm, PyPI, and crates.io packages," arXiv:2607.15315, 2026, ICSME 2026 Tool Demonstration and Data Showcase Track. [Online]. Available: https://arxiv.org/abs/2607.15315

[12] A. A. Shafin and K. M. Ahmed, "PatchRisk: Forecasting future vulnerability exposure in open-source dependency networks," arXiv:2609.13719, 2026, WI-IAT 2026 preprint. [Online]. Available: https://arxiv.org/abs/2609.13719

[13] Open Source Insights, "deps.dev API v3: GetDependencies," Official API documentation, accessed October 20, 2026. [Online]. Available: https://docs.deps.dev/api/v3/